Deadlock

- Condition for deadlock
  - A P/T requires an unavailable resource, it enters a waiting state, and it waits forever
- Deadlock consists in
  - A set of P/T all awaiting the occurrence of an event that can only be caused by another process in the same set
- Deadlock implies starvation, not the opposite
  - The starvation of a P/T implies that this P/T waits indefinitely, but the other P/T can proceed in the usual way (without being in deadlock)
  - All P/T in deadlock are in starvation

Deadlock problem

- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.
  - Example: P_1 and P_2
    - each of them holds a pen drive and
    - needs another one.
- Solution with 2 semaphores A and B, initialized to 1

P_1	P_2
wait (A)	wait(B)
wait (B)	wait(A)

Necessary conditions for occurrence of a deadlock

Conditions	Description
Mutual exclusion	Only one process at a time can use a **not sharable** resource
Hold and wait	A process **holding** at least one resource is allowed to **wait** for acquiring additional resources held by other processes
No preemption	A resource can be released only voluntarily by the process holding it, **cannot** be **preempted** by the system.
Circular wait	A set of waiting processes $\{P_1, P_2, \ldots, P_n\}$ such that P_1 is **waiting** for a resource that is held by P_2, P_2 is **waiting** for a resource that is held by P_3, ..., and P_n is **waiting** for a resource that is held by P_1

All must occur simultaneously to have a deadlock

Necessary but not sufficient conditions. They are distinct but not independent (e.g., 4→2)

Deadlock modeling

Resource allocation graph G = (V, E). It allows for deadlock description and analysis.

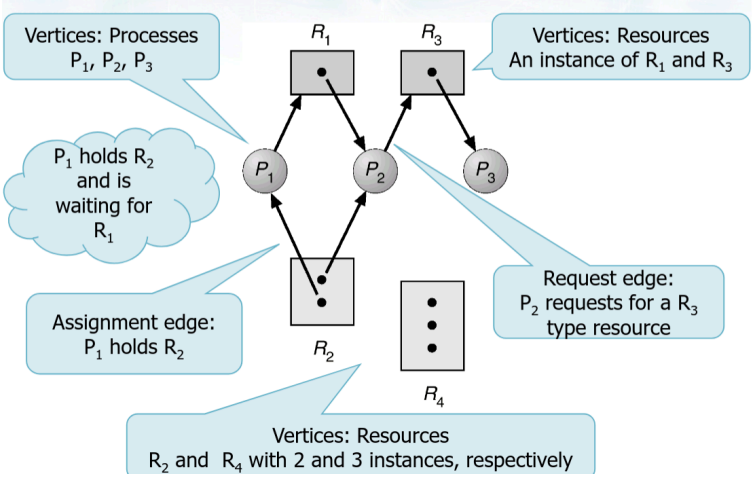
The set of vertices V is composed of processes and resources:

- Process set P = {P_1, P_2, ...,P_n}
  - Processes are indistinguishable and in an indefinite number
  - Each process accesses a resource via a standard protocol consisting of
    - Request
    - Utilization
    - Release
- System resource set R = {R_1, R_2, …, Rm}

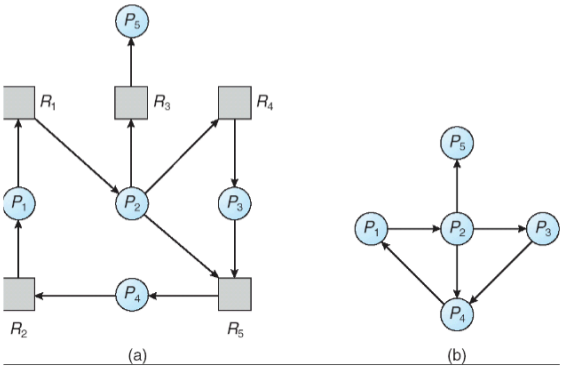
- The resources are divided into classes (types)
- Each resource type Rj has Wi instances
- All instances of a class are identical: any instance satisfies a demand for that type of resource

The set of edges E is composed of

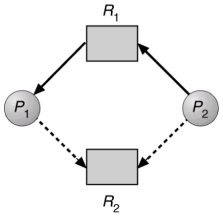
- Request edges
  - $P_i \rightarrow R_j$, i.e., from a process to a resource type
- Assignment edge
  - $R_j \rightarrow P_i$, i.e., from a resource to a process



- A resource allocation graph can be sometime simplified in a wait-for graph by
  - deleting the resource vertices
  - creating the edges between the remaining vertices
- Use and consideration similar to the resource allocation graph



- Sometimes it is useful to extend the resource-allocation graph to a claim graph by
  - adding a claim edge: P_i -> R_j , indicates that process P_j can ask resource R_j in the future
  - A claim arc is represented by dashed line



Detection and recovery techniques

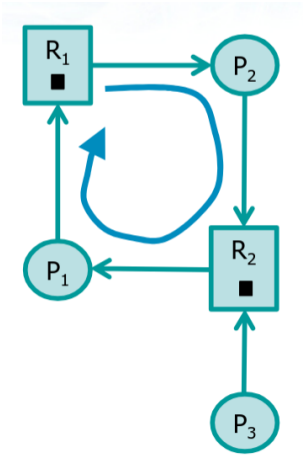
- The system is allowed to enter in a deadlock state, to then intervene.
- Algorithm in two steps
  - Deadlock detection (of deadlock condition)
    - The system performs a deadlock detection algorithm
  - Recovery from deadlock
    - If deadlock has been detected, a recovery action is performed

Detection: strategies

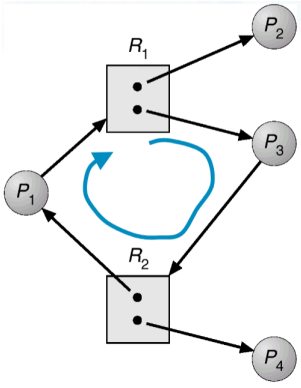
- Given an allocation graph, deadlock can be detected by checking for cycles
  - If the graph contains no cycles, then there is no deadlock
  - If the graph contains one or more cycles then
    - Deadlock exist if each type of resource has a single instance
    - Deadlock is possible if the are several instances per resource type
      - The presence of cycles is necessary but not sufficient condition in the case of multiple instances per resource type (For multiple instances see the Banker's Algorithm)

Examples:

- Processes
  - P_1, P_2, P_3
- Resources
  - R_1 and R_2 with a single instance
- A cycle exists
- Deadlock
  - P_1 waits for P_2
  - P_2 waits for P_1

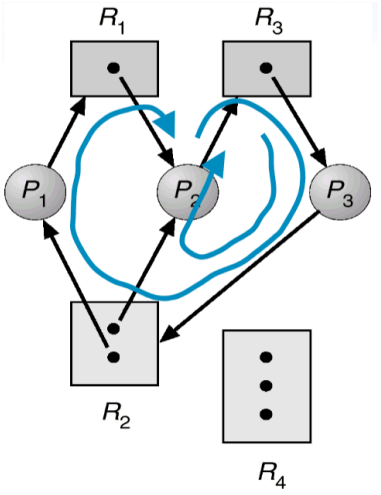


- Processes
  - P1, P2, P3, P4
- Resources
  - R1 and R2 with two instances
- A cycle exists
- No deadlock
  - P2 and P4 can terminate
  - P1 can acquire R1 and terminate
  - P3 can acquire R2 and terminate



- Processes
  - P1, P2, P3

- Resources
  - R1 and R3 with an instance
  - R2 with two instances
  - R4 with three instances
- Two cycles exist
- Deadlock
  - P1 waits for R1
  - P2 waits for R3
  - P3 waits for R2



Detection costs

- The detection phase has the high computational cost
  - An algorithm to detect a cycle in a graph is required
  - The presence of cycles can be verified by a visit in depth
  - A graph is acyclic if a visit in depth does not meet arcs labeled "backward" directed to gray vertices
    - If you reach a gray vertex, i.e., you cross a backward arc, you have a cycle
  - The computational cost of this operation is equal to
    - $\Theta(|V| + |E|)$ for representations with adjacency list
    - $\Theta(|V|^2)$ for representations with adjacency matrix
- When detection is performed?
  - Every time a process makes a request not immediately satisfied
  - At fixed time intervals, e.g., every 30 minutes
  - At variable intervals of time, e.g., when the CPU usage falls below a given threshold

Recovery

- Different strategies are possible for deadlock recovery
  - Act on the vertex of allocation graphs
  - Act on the arches of allocation graph

Strategy	Description
Terminate all deadlocked processes	• **Complexity**: **low**, but easy to cause inconsistencies on databases • **Cost**: **much higher than it might be** strictly **necessary**
Terminate a process at a time among the ones in deadlock	• **Complexity**: **high**, since it is necessary to select the victims with objective criteria (priority, current and future execution time, number of held resources, etc.) • **Cost**: **high**, after each termination must re-check the deadlock condition
Preempt the resources of a deadlocked process at a time	• **Complexity**: rollback is necessary to return the selected process to a safe state • **Cost**: the victim process selection must aim at minimizing the preemption cost

Strategy	Description
Remove **holding** arcs (i.e., specific resources)	• **Complexity**: rollback is necessary to return the selected process to a safe state. The arc must be properly selected. • **Cost**: the victim process selection must aim at minimizing the preemption cost. • Same as preemption strategy.
Remove **waiting** arcs	• **Complexity**: The arc must be properly selected. • **Cost**: the victim must manage only the failure of a resource request (e.g., a malloc that returns with an error message).

Best strategy

Conclusion on Detection and Recovery

- Detection and recovery operations are
  - logically complex
  - computationally expensive
- In any case, if a process requires many resources, starvation may occur
  - The same process is repeatedly chosen as the victim, incurring repeated rollbacks
    - To avoid starvation the victim selection algorithm should take into account the number of a process rollbacks

Prevention techniques

Try to control how resources are requested to prevent the occurrence of at least one of the necessary conditions

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Mutual exclusion

A deadlock occurs because of "mutual exclusion" when a process is indefinitely waiting a non sharable resource.	
Thus, deadlock could be avoided if 1. All resources were shareable (e.g., read-only) 2. A process could not wait for a resource not immediately available	
Strategy 1	• Allow only shareable resources. • This strategy is generally considered very restrictive.
Strategy 2	• Inhibit a process to wait for a resource that is not immediately available. • This strategy is considered complex to be implemented

Hold and wait

A deadlock occurs because of a "hold and wait" condition, where a process requests further resources while holding one or more resources.	
So a hold and wait condition can be avoided by imposing that a process waits for a resource only when it does not hold others	
Request All First (RAF)	A process must acquire all the necessary resources before starting its processing activities • Poor resource usage • Resources may be assigned a long time in advance of their usage
Release Before Request (RBR)	A process can request resources only when it has not previously acquired other resources • Before each new request each process must release the resources already held • Possibility of starvation • Processes requiring many widely used resources may have to "start over" very often

No preemption

A deadlock occurs because no preemption is possible of a resource held by a process	
In general is not easy to divert resources from a running process, but a similar effect can be obtained by means of the following strategies:	
Allow preemption of resources held by the process itself	• If a process that holds some resources asks for another that cannot be immediately granted, it is forced to release all held resources (preemption). • These resources are added to the list of resources that the process is waiting for. • The process will be awakened only when it can regain its old resources, and additionally the new one.
A deadlock occurs because no preemption is possible of a resource held by a process	
Allowing preemption of resources owned by another process as long as it is waiting	• If process P asks for a resource that is not immediately available, a search is performed for the process that currently holds it • If a process Q is found, which is waiting for another resource, preempt from Q the resource and assign it to process P • Otherwise, process P goes on the waiting state, so that the resources it hold can be preempted
Both strategies • are suited for resources whose state can be easily saved and restored (CPU registers, main memory, etc.) • are not suited for resources whose state cannot be recovered (files, printers, etc.)	

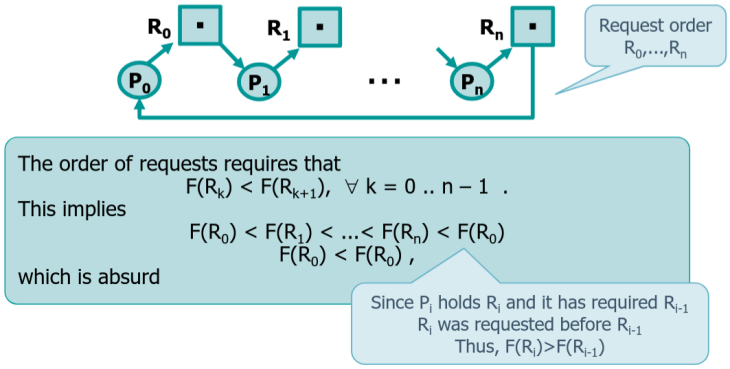
Circular wait

A deadlock occurs because of a "circular wait" when a set of processes is waiting for a resource held by another set o processes	
To avoid this condition, one can impose a total ordering of all resource classes	
Hierarchical Resource Usage (HRU)	• It imposes a total ordering relation between the various types of resources, associating to each of them an integer number. Example: HD = 1, DVD = 5, printers = 12 • Force each process to request resources with an increasing order of enumeration In general, the HRU verification is applied by • Programmer • Operating system. The **`witness`** tool, available in FreeBSD UNIX version, checks the order of the lock acquired by processes

Let F be the function that imposes a unique order among all classes of system resources R_i

- Let a process have previously requested an instance of R_{old} resource, and now request a R_{new} instance
- If $F(R_{new}) > F(R_{old})$
  - The resource is granted
- If $F(R_{new}) \leq F(R_{old})$
  - The process must release all resources R_i such that $F(R_{new}) \leq F(R_i)$ before getting an instance of R_{new}
- It can be shown that this condition is sufficient to avoid the circular wait
  - That is, if the resources are requested in a certain order, is it true that it is not possible to have a circular wait?
  - We proceed using a demonstration of type "reduction to absurdity", assuming there is a circular wait, i.e., supposing there is a set of processes that
    - They were requested in the specified order, e.g., in increasing numerical order

- They are in circular wait
  - ❖ Let's suppose that there exists a set of processes that satisfy the HRU rules and are in circular wait

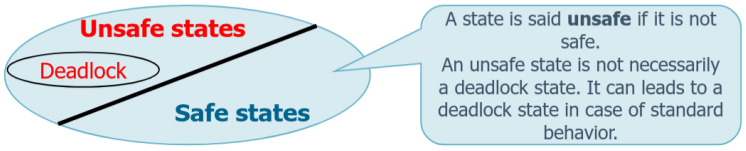


Deadlock avoidance

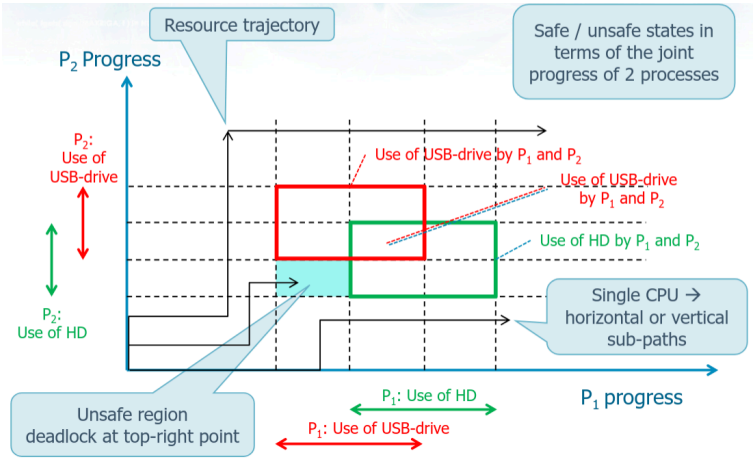
- The main algorithms
  - differ in the amount and type of information required
    - The simplest model imposes that all processes declare the maximum number of resources of each type that they will need
  - generally reduce the use of resources and the efficiency of the system
  - are based on the concept of safe state and safe sequence

Safe state

Safe state	**The system is able to** **• Allocate the required resources to all processes** **• Prevent the occurrence of a deadlock** **• Find a safe sequence**
Safe sequence	A sequence of process scheduling {P1, P2, ...,Pn} such that for each requests that could be performed by any Pi, the request can be satisfied by using the currently available resources and the other resources released by processes Pj with j < i



Joint progress of two processes



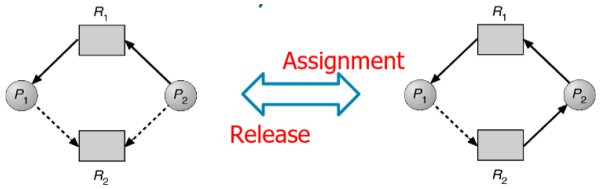
Strategies

- To avoid a deadlock, one must ensure that the system remains always in a safe state

    - Initially the system is in a safe state
    - Each new resource request
        - will be granted immediately, if this allows the system to remain in a safe state
        - otherwise, granting the request will be delayed; the process that performed the request is forced to wait
- There are two classes of strategies
    - For resources having unitary instances
    - For resources having multiple instances

Algorithm for resources with a single instance

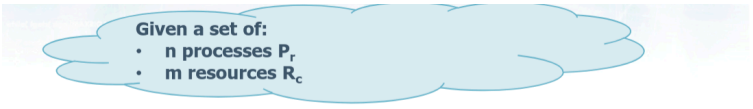
- Based on the determination of cycles, using the claim-for graph
    - All requests must be a priori declared
    - they are represented by claim arcs
- At a time a request is performed
    - the corresponding claim arc is transformed into an assignment arc
    - Before the request is satisfied, the algorithm verify the presence of cycles
    - If no cycle is present, the conversion of the arc is performed and the resource assigned
    - Otherwise, the assignment of the requested resource would bring the system into an unsafe state. For this reason it is postponed
- Each time a resource is released
    - the assignment arc is transformed into a claim arc (to manage any subsequent request)



Algorithm for resources with a multpiple instance

- Verify the state of the system to understand if the available resources are sufficient to complete all processes based on
    - the number of resources available to the system
    - number of resources allocated, and
    - max number of resource that the process may need
- Each process
    - must declare in advance its maximum number of resources it may need
    - when it requests a resource, it can be blocked for a limited amount of time
    - must guarantee to return an allocated resource in a finite amount of time
- Banker's Algorithm (Dijkstra, [1965])
    - It consists of two parts
        - Verifies that the current state is safe
        - Verifies whether the new request can be immediately granted allowing to system to remain in a safe state
            - Simulates assigning the resource, and controls that a sequence of assignments exists that allows the system to satisfy all requests, possibly delaying the delivery of the resources for some of the requests.

- The algorithm uses the data structures listed in the following slide



Name	Dim.	Content and meaning
finish	[n]	finish[r] initially false (indicates P_r has not compete)
allocation	[n][m]	allocation[r][c]=k P_r owns k instances of R_c
max	[n][m]	max [r][c]=k P_r can ask a maximum of k instances of R_c
need	[n][m]	need[r][c]=k P_r needs k additional instances of R_c $\forall i \forall j$ need[i][j]=max[i][j]-allocation[i][j]
available	[m]	available[c]=k k resources R_c are available

Example:

- By applying the banker algorithm, the underlying system is in a safe state?
  - Safe sequence: P_1, P_3, P_0, P_2, P_4

P	finish	allocation	max	need	available
		R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2
P_0	F	0 1 0	7 5 3		3 3 2
P_1	F	2 0 0	3 2 2		
P_2	F	3 0 2	9 0 2		
P_3	F	2 1 1	2 2 2		
P_4	F	0 0 2	4 3 3		

- Can the request of P_1 (1, 0, 2) be satisfied?
  - Yes ...
  - System state evolution ...

P	finish	allocation	max	need	available
		R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2
P_0	F	0 1 0	7 5 3	7 4 3	3 3 2 2 3 0
P_1	F	2 0 0 3 0 2	3 2 2	1 2 2 0 2 0	
P_2	F	3 0 2	9 0 2	6 0 0	
P_3	F	2 1 1	2 2 2	0 1 1	
P_4	F	0 0 2	4 3 3	4 3 1	

❖ The new state is safe or not?
➢ Safe sequence: P_1, P_3, P_0, P_4, P_2

P	finish	allocation	max	need	available
		R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2
P_0	F	0 1 0	7 5 3	7 4 3	2 3 0
P_1	F	3 0 2	3 2 2	0 2 0	
P_2	F	3 0 2	9 0 2	6 0 0	
P_3	F	2 1 1	2 2 2	0 1 1	
P_4	F	0 0 2	4 3 3	4 3 1	

❖ Can the request of P_4 (3, 3, 0) be satisfied?
➢ No ... there is not availability (-> wait)
❖ Can the request of P_0 (0, 3, 0) be satisfied?
➢ No ... the resulting state is not safe

P	finish	allocation	max	need	available
		R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2	R_0 R_1 R_2
P_0	F	0 1 0	7 5 3	7 4 3	2 3 0
P_1	F	3 0 2	3 2 2	0 2 0	
P_2	F	3 0 2	9 0 2	6 0 0	
P_3	F	2 1 1	2 2 2	0 1 1	
P_4	F	0 0 2	4 3 3	4 3 1	

Banker's algorithm

Verify whether a state is safe or unsafe

1.

```
∀i∀j need[i][j]= max[i][j] - allocation[i][j]
∀i finish[i]=false
```

2.

```
Find a process Pi such that finish[i]=false AND ∀j need[i][j] <= available[j]
If no such i is found goto step 4
```

3.

```
∀j available[j] += allocation[i][j]
finish[i]=true
goto step 2
```

4.

```
if ∀i finish[i]=true then system is in a safe state
```

- Complexity is
  - $O(m \cdot n^2) = O(|R| \cdot |P|^2)$
- It is also based on unrealistic assumptions
  - Processes must specify their demands in advance
    - The necessary resources are not always known
    - Also it is not known when a resource will be used
  - Assumes that the number of resources is constant
    - Resources may increase or decrease due to transient or continuous failures
  - It requires a fixed population of processes

- The number of active processes in the system increases and decreases dynamically